

Sprint 2 Implementation Plan

PrepAI - AI Interviewer Application

Overview

This document outlines the technical implementation plan for Sprint 2, detailing the complete flow from user input through backend processing, AI interaction, and data storage.

System Flow Diagram

1. Client Entry & Input Form

User Action:

- Client enters the application and views the input form

Required Information:

- Resume (file upload)
 - Job Name
 - Job Title
 - Job Description
-

2. Data Storage Layer

Database 1: User Information Database

Purpose: Store user profile and job-related information

Stored Data:

- User resume
- Job name
- Job title
- Job description

- User profile metadata

Technical Notes:

- Primary storage for candidate information
 - Enables personalized interview generation
-

Database 2: Evaluation Database

Purpose: Store interview session data for analysis and feedback

Stored Data:

- Interview questions generated by AI
- User answers/responses
- Evaluation metrics and scores
- Session timestamps

Technical Notes:

- Supports post-interview feedback generation
 - Maintains interview history for improvement tracking
-

3. Backend Processing Flow

Step 1: Receive User Input

- Backend receives information from frontend form submission
- Validates and processes uploaded resume
- Extracts structured data from job description

Step 2: AI Prompt Generation

- Backend constructs a comprehensive prompt for the AI using:
 - Parsed resume content
 - Job title and description
 - Company-specific interview style (if applicable)
 - Interview type selection (technical/behavioral/full)

Step 3: AI Question Generation

- Send prompt to AI service (Gemini API)

- AI generates contextually relevant interview questions based on:
 - Candidate's experience and skills from resume
 - Target role requirements from job description
 - Selected company's interview patterns

Step 4: Question Delivery

- Backend receives AI-generated questions
 - Formats questions for frontend display
 - Sends questions to frontend for presentation
-

4. Frontend Interview Interaction

Display Interview Questions

- Frontend receives questions from backend
- Presents questions one at a time to the user
- Provides interface for user to input answers (text/code editor)

Continuous Answer Collection

- User provides answers through the interface
- Frontend sends each answer back to backend in real-time
- Backend forwards answers to AI with modified evaluation prompt

AI Response Loop

- AI analyzes user's answer
- Generates follow-up questions or moves to next question
- Provides hints or clarification if needed
- Continues conversational interview flow

Data Persistence

- All questions and answers are simultaneously stored in the Evaluation Database
 - Enables post-interview analysis and feedback
-

5. Backend Session Management

Critical Implementation Note:

The backend must maintain continuous AI session state throughout the interview. This is challenging with standard REST API patterns.

Technical Considerations:

- **Session Continuity:** The entire interview conversation must be memorized by the AI
- **Context Preservation:** Each new question/answer exchange builds on previous interactions
- **Implementation Options:**
 - WebSocket connection for real-time bidirectional communication
 - Session-based API with conversation history management
 - Stateful backend service maintaining AI conversation context

Challenge: Regular REST API calls lose conversation context between requests, making it difficult to maintain interview flow continuity.

Technical Requirements Summary

Frontend Requirements

- Input form for resume upload and job details
- Real-time question/answer interface
- Monaco code editor integration (for coding questions)
- WebSocket or polling mechanism for continuous communication

Backend Requirements

- FastAPI routes for form submission and interview orchestration
- Resume parsing service (PDF.js or similar)
- AI service integration (Gemini API)
- Session management for maintaining interview state
- Database connections for User Info and Evaluation databases

Database Requirements

- **User Info Database:** User profiles, resumes, job preferences
- **Evaluation Database:** Interview questions, answers, scores, feedback

AI Integration Requirements

- Prompt engineering for question generation
 - Conversation context management
 - Real-time response processing
 - Evaluation and feedback generation
-

Sprint 2 Deliverables

1. **Functional input form** with resume upload and job details
 2. **Backend API routes** for data processing and AI orchestration
 3. **Database schemas and connections** for User Info and Evaluation storage
 4. **AI question generation** via Gemini API with contextual prompting
 5. **Real-time interview flow** (text-to-text interaction as MVP)
 6. **Basic post-interview feedback** generation from stored evaluation data
-

Next Steps

- Finalize database schema design
 - Implement WebSocket or session-based architecture for continuous AI interaction
 - Create prompt templates for different interview types
 - Build frontend interview UI with code editor
 - Test end-to-end flow with sample resumes and job descriptions
-

Document Version: 1.0

Date: March 4, 2026

Team: Final Round - UTA CSE Senior Design